

Instalação Automática do Ollama - Documentação

Resumo

Sistema completo de instalação automática do Ollama integrado ao Assistente Xiaozhi AI. Detecta o sistema operacional, instala o Ollama, inicia o serviço e faz download do modelo LLaVA automaticamente.

Arquivos Criados

1. `setup_ollama.py` (423 linhas)

Descrição: Script Python multiplataforma para instalação automática do Ollama.

Funcionalidades:

☒ **Deteção Automática de SO:**

- Windows: Download do instalador .exe oficial
- macOS: Instalação via Homebrew ou script shell
- Linux: Script de instalação oficial com sudo

☒ **Gerenciamento de Serviço:**

- Verifica se Ollama está instalado (`ollama --version`)
- Verifica se serviço está rodando (`http://localhost:11434`)
- Inicia serviço automaticamente se necessário
- Aguarda inicialização com feedback visual

☒ **Download de Modelo:**

- Baixa modelo LLaVA (7B) automaticamente
- Mostra progresso do download em tempo real
- Validação de instalação completa

☒ **Interface Colorida:**

- Mensagens verde (☑) para sucesso
- Mensagens amarela (⚠) para avisos
- Mensagens vermelha (✖) para erros
- Banner azul com título

☒ **Verificação Final:**

- Checa instalação do Ollama
- Valida serviço rodando
- Confirma modelo LLaVA disponível

- Relatório completo de status

Uso:

```
python setup_ollama.py
```

Fluxo:

1. Verifica se Ollama já está instalado
 - └ SIM: Verifica serviço e modelo
 - └ NÃO: Prossegue com instalação
2. Detecta Sistema Operacional
 - └ Windows: Baixa OllamaSetup.exe
 - └ macOS: Usa Homebrew ou curl
 - └ Linux: Script oficial com sudo
3. Inicia Serviço
 - └ Windows: ollama serve em nova console
 - └ macOS: ollama serve em background
 - └ Linux: systemctl ou processo direto
4. Baixa Modelo LLaVA
 - └ ollama pull llava:7b (4.5GB)
5. Verificação Final
 - └ ✓ Ollama instalado e versão
 - └ ✓ Serviço ativo na porta 11434
 - └ ✓ Modelo llava disponível

2. **start.bat** (85 linhas)

Descrição: Launcher para Windows com verificação integrada.

Funcionalidades:

☒ Gerenciamento de Ambiente:

- Detecta e ativa ambiente virtual (.venv-1 ou venv)
- Verifica Python instalado
- Mensagens de status claras

☒ Verificação Ollama:

- Detecta se Ollama está instalado
- Oferece instalação automática interativa
- Inicia serviço se necessário

- Verifica modelo LLaVA

☒ Inicialização do Assistente:

- Executa `python main.py --mode gui --protocol websocket`
- Captura e reporta erros
- Pausa em caso de falha

Uso:

```
start.bat
```

Exemplo de Output:

```
=====
XIAOZHI AI ASSISTANT - STARTUP
=====

[INFO] Ativando ambiente virtual...
[INFO] Verificando Ollama...
[OK] Ollama instalado
[OK] Modelo LLaVA disponível

=====
INICIANDO ASSISTENTE
=====
```

3. `start.sh` (94 linhas)

Descrição: Launcher para Linux/macOS com verificação integrada.

Funcionalidades:

☒ Interface Colorida:

- Usa códigos ANSI para cores
- Feedback visual profissional
- Alinhado com `setup_ollama.py`

☒ Gerenciamento de Ambiente:

- Ativa ambiente virtual automaticamente
- Verifica `python3` disponível
- Compatível com `bash/zsh`

☒ Verificação Ollama:

- Usa `command -v` para detecção
- Instalação interativa com confirmação
- Testa conectividade HTTP
- Valida modelo com `grep`

☑ Execução do Assistente:

- `python3 main.py --mode gui --protocol websocket`
- Código de saída apropriado
- Mensagens de erro coloridas

Uso:

```
chmod +x start.sh
./start.sh
```

4. Modificações em `main.py`

Função Adicionada: `check_ollama_setup()` (linha 12-94)

Integração:

```
async def start_app(mode: str, protocol: str, skip_activation: bool) -> int:
    logger.info("Iniciando Cliente AI Xiaozhi")

    # ✨ NOVA VERIFICAÇÃO
    logger.info("\n🔍 Verificando dependências do sistema...")
    check_ollama_setup()

    # ... resto do código
```

Comportamento:

1. Ollama Instalado e Rodando:

☑ Ollama e LLaVA configurados corretamente

2. Ollama Instalado, Serviço Parado:

⚠ Serviço Ollama não está rodando
Iniciando serviço Ollama...
☑ Serviço Ollama iniciado

3. Modelo LLaVA Ausente:

⚠ Modelo LLaVA não encontrado
Para análise de imagens, execute: `python setup_ollama.py`

4. Ollama Não Instalado:

```
=====
❌ OLLAMA NÃO INSTALADO
=====

O Ollama é necessário para análise de imagens (100% gratuito).

Opções:
  1. Instalação automática: python setup_ollama.py
  2. Instalação manual: https://ollama.ai/download

=====

Deseja instalar o Ollama automaticamente agora? (s/N):
```

Dependência Adicionada:

```
import subprocess # Para executar comandos ollama
```

5. Atualização do README.md

Seção Adicionada: "Instalação" (linhas 794-900)

Estrutura:

🔗 Instalação Rápida (Recomendada)

- Scripts `start.bat` / `start.sh`
- Verificação automática de tudo
- Zero configuração manual

🔗 Instalação Manual Completa

1. Clonar repositório
2. Ambiente virtual Python
3. Instalar dependências
4. **Instalar Ollama** (Opção A: automática | Opção B: manual)
5. Executar aplicação

Instalação Sem Ollama

- Para quem não precisa de visão computacional
- Instalação mínima apenas Python

Verificar Instalação

- Comandos para testar Python, Ollama e sistema

Design Patterns Utilizados

1. Factory Pattern

Script detecta SO e chama função apropriada:

```
if system == "Windows":
    success = install_ollama_windows()
elif system == "Darwin":
    success = install_ollama_macos()
elif system == "Linux":
    success = install_ollama_linux()
```

2. Strategy Pattern

Múltiplas estratégias de instalação:

- Homebrew vs Script Shell (macOS)
- systemctl vs Processo Direto (Linux)
- Instalador GUI vs Linha de Comando (Windows)

3. Observer Pattern

Progress tracking com callbacks:

```
for line in process.stdout:
    print(f" {line.strip()}") # Observa output do ollama
```

4. Template Method

Fluxo comum com implementações específicas:

```
def main():
    print_banner()           # Comum
    detect_system()         # Comum
    install_platform()       # Específico
```

```
start_service()      # Comum
download_model()     # Comum
verify_installation() # Comum
```

Segurança

Validações Implementadas

☒ Verificação de Comandos:

```
result = subprocess.run(..., timeout=5) # Timeout de 5s
if result.returncode == 0: # Valida sucesso
    # Prossegue
```

☒ Tratamento de Exceções:

- `FileNotFoundError`: Comando não existe
- `subprocess.TimeoutExpired`: Comando travou
- `ConnectionError`: Serviço não responde
- `KeyboardInterrupt`: Usuário cancelou

☒ Privilégios Mínimos:

- Linux: sudo apenas quando necessário
- Windows: Instalador executa com privilégios usuário
- macOS: Homebrew sem sudo

☒ URLs Oficiais:

- <https://ollama.ai/download>
- <https://ollama.ai/install.sh>
- Sem redirecionamentos ou CDNs terceiros

Estatísticas

Métrica	Valor
Linhas de Código	423 (setup_ollama.py)
Funções	14 funções principais
Plataformas Suportadas	3 (Windows, Linux, macOS)
Verificações de Saúde	3 (instalado, rodando, modelo)
Tamanho Download	~4.5 GB (modelo LLaVA)

Métrica	Valor
Tempo Instalação	5-15 min (depende da internet)

Testes

Como Testar

1. Teste Completo (Sistema Limpo):

```
# Desinstalar Ollama primeiro
# Windows: Desinstalar via Painel de Controle
# Linux: sudo rm /usr/local/bin/ollama
# macOS: brew uninstall ollama

# Executar instalação
python setup_ollama.py
```

2. Teste de Atualização (Ollama Já Instalado):

```
python setup_ollama.py
# Deve detectar instalação existente e verificar modelo
```

3. Teste de Serviço Parado:

```
# Parar serviço manualmente
# Linux: sudo systemctl stop ollama
# macOS/Windows: pkill ollama

# Executar script
python main.py --mode gui
# Deve detectar e reiniciar serviço
```

4. Teste dos Launchers:

```
# Windows
start.bat

# Linux/macOS
./start.sh
```

Troubleshooting

Problema: "Ollama não encontrado após instalação"

Solução:

```
# Adicionar ao PATH manualmente
# Windows: Editar variáveis de ambiente
# Linux/macOS: export PATH="/usr/local/bin:$PATH"
```

Problema: "Erro ao baixar modelo LLaVA"

Solução:

```
# Verificar espaço em disco
df -h # Linux/macOS
wmic logicaldisk get size,freespace # Windows

# Verificar conectividade
curl -I https://ollama.ai

# Download manual
ollama pull llava:7b
```

Problema: "Serviço não inicia automaticamente"

Solução:

```
# Iniciar manualmente
ollama serve

# Verificar portas
netstat -an | grep 11434
lsof -i :11434 # Linux/macOS
```

Problema: "Timeout ao verificar Ollama"

Solução:

```
# Aumentar timeout em check_ollama_setup()
result = subprocess.run(..., timeout=10) # Era 5
```

Próximos Passos

Melhorias Futuras

- ☐ **Auto-update do Ollama:** Detectar versões novas
- ☐ **Seleção de Modelo:** Permitir escolher llava:7b, 13b ou 34b
- ☐ **Cache de Instalador:** Salvar .exe para reinstalações
- ☐ **Instalação Silenciosa:** Flag `--silent` para CI/CD
- ☐ **Verificação de GPU:** Detectar CUDA/ROCm para aceleração
- ☐ **Múltiplos Modelos:** Instalar outros além do LLaVA
- ☐ **Rollback:** Desinstalar/reverter se algo falhar
- ☐ **Telemetria:** Log de instalações bem-sucedidas

Referências

- **Ollama Oficial:** <https://ollama.ai>
- **LLaVA Model:** <https://llava-vl.github.io>
- **GitHub Actions:** Para CI/CD futuro
- **PyInstaller:** Para criar executável standalone

☒ Checklist de Conclusão

- ☒ Script multiplataforma criado
- ☒ Detecção de SO implementada
- ☒ Instalação automática funcional
- ☒ Gerenciamento de serviço
- ☒ Download de modelo automatizado
- ☒ Verificação de saúde completa
- ☒ Integração com main.py
- ☒ Launchers Windows/Linux criados
- ☒ README atualizado
- ☒ Documentação completa
- ☒ Commits no GitHub
- ☒ Testes realizados

Comandos Rápidos

```
# Instalação automática completa
python setup_ollama.py

# Verificar instalação
ollama --version
ollama list
curl http://localhost:11434/api/tags

# Iniciar aplicação
python main.py --mode gui
```

```
# Ou usar launchers
start.bat          # Windows
./start.sh         # Linux/macOS

# Diagnóstico completo
python diagnose_system.py

# Reinstalar modelo
ollama pull llava:7b

# Atualizar Ollama
# Windows: Baixar novo instalador
# Linux: curl -fsSL https://ollama.ai/install.sh | sh
# macOS: brew upgrade ollama
```

Criado em: 13 de Janeiro de 2026

Versão: 1.0.0

Autor: GitHub Copilot

Repositório: <https://github.com/MarceloClaro/ASSISTENTE-SHI>